

Ensemble Methods

Aprendizagem Computacional Ciências da Computação

Agenda

- Motivation
- Bagging & Random Forests
- Boosting
- Stacking
- Practical considerations

Section 1

Motivation

Motivation for Ensembles I

- No single model is perfect in practice
- Real-world analogy: combining forecasts from multiple experts

Motivation for Ensembles II

- Goal: improve robustness and generalization
- Intuition: “crowd wisdom” effect
- Diagram idea: individual model errors vs averaged prediction

Section 2

Bias-Variance Tradeoff

Introduction: Predictive Error

Why do our models fail? We can decompose the total error of any supervised learning model into three parts:

- **Bias:** Error due to overly simplistic assumptions.
- **Variance:** Error due to sensitivity to fluctuations in the training set.
- **Irreducible Noise (σ^2):** Inherent uncertainty in the data that no model can capture.

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

Defining Bias

- **Concept:** The difference between the average prediction of our model and the true value we are trying to predict.
- **High Bias:**
 - ▶ The model makes strong, incorrect assumptions.
 - ▶ Leads to **underfitting**.
 - ▶ Examples: Using linear regression for a highly non-linear relationship.
- **Intuition:** A “stubborn” model that ignores the data trends.

Defining Variance

- **Concept:** The variability of the model prediction for a given data point if we were to train it on different training sets.
- **High Variance:**
 - ▶ The model is extremely sensitive to small changes in the training data.
 - ▶ Leads to **overfitting**.
 - ▶ The model memorizes noise as if it were signal.
- **Intuition:** A “fickle” model that overreacts to every data point.

The Tradeoff Visualized

As we increase model complexity (e.g., adding features, increasing polynomial degree, or tree depth):

- ➊ **Bias decreases:** The model becomes flexible enough to capture the underlying pattern.
- ➋ **Variance increases:** The model becomes flexible enough to capture the noise.
- **Goal:** Find the “sweet spot” where total error is minimized.

Dartboard Analogy

Imagine predicting the bullseye on a dartboard:

- **Low Bias, Low Variance:** Tightly clustered around the bullseye. (Ideal)
- **High Bias, Low Variance:** Tightly clustered, but far from the bullseye. (Underfit)
- **Low Bias, High Variance:** Spread out, but centered around the bullseye. (Overfit)
- **High Bias, High Variance:** Spread out and far from the bullseye. (Worst case)

Practical Strategies

How do we balance the tradeoff?

- **To reduce High Bias (Underfitting):**

- ▶ Increase model complexity (e.g., more layers, more features).
- ▶ Reduce regularization.
- ▶ Use more advanced algorithms (e.g., Gradient Boosting vs. Linear Regression).

- **To reduce High Variance (Overfitting):**

- ▶ Get more training data.
- ▶ Increase regularization (e.g., L1/L2 penalty).
- ▶ Use ensemble methods (e.g., Random Forest or Bagging).
- ▶ Perform early stopping.

Section 3

Ensembles

What is an Ensemble?

- **Core Concept:** Combining multiple “base learners” to create a single, stronger predictive model.
- **The Goal:** To reduce **variance** (overfitting) and **bias** (underfitting).
- **The Analogy:** Aggregating diverse opinions usually leads to a more accurate conclusion than relying on one expert.

Ensemble Taxonomy I

- Homogeneous vs heterogeneous ensembles
 - ▶ Homogeneous: same model type (e.g., many trees)
 - ▶ Heterogeneous: different model families (SVM, tree, NN)
- Parallel vs sequential training

Homogeneous Ensembles

The “Same-Algorithm” Approach

- **Definition:** Every base learner uses the **same** underlying machine learning algorithm.
- **Example: Random Forest** (composed of hundreds of Decision Trees).
- **The Challenge:** Since they use the same “logic,” they may all make the same systematic errors.
- **How they succeed:** We force diversity through:
 - ▶ **Bagging:** Training identical models on different random subsets of data.
 - ▶ **Boosting:** Training models sequentially where each new model fixes the errors of the previous one.

Heterogeneous Ensembles

The “Diverse-Algorithm” Approach

- **Definition:** Combines models built on **entirely different** mathematical foundations.
- **The Lineup:** You might mix a *Logistic Regression*, a *Support Vector Machine (SVM)*, and a *K-Nearest Neighbor (KNN)*.
- **The Strength:** Different algorithms have different “blind spots.” One model’s weakness is covered by another’s strength.
- **Implementation:** Usually handled via **Stacking**, where a “Meta-Learner” decides how much to trust each individual model’s output.

Comparison Table

Feature	Homogeneous	Heterogeneous
Base Models	Identical (e.g., all Trees)	Diverse (e.g., SVM)
Diversity Source	Data sampling (Bagging/Boosting)	Algorithmic differences
Complexity	Lower (one set of parameters)	Higher (multiple models)
Typical Use	High-performance “workhorses”	Squeezing out final performance

Summary: Which to Choose?

- **Use Homogeneous if:** You want a robust, well-understood model that is relatively easy to tune (e.g., **XGBoost** or **Random Forest**).
- **Use Heterogeneous if:** You are in a high-stakes competition or research environment where marginal gains in accuracy are worth the extra computational complexity.

Combination Strategies

Voting

- Used in classification
- Each classifier outputs a class
- The class with the most votes wins
- The voting process may be weighted

Averaging

- Used in regression
- Each regressor predicts a value
- The final result is either an average or a weighted average

Stacking

- There are two levels: Level 0 and Level 1
- Each Level 0 classifier or regressor outputs a prediction
- The Level 1 classifier or regressor uses the predictions from the Level 0 classifiers in order to output its prediction

Section 4

Bagging

Bagging: Bootstrap Aggregation I

- Training procedure:
 - ▶ Sample with replacement from training data
 - ▶ Train one base learner per bootstrap sample
 - ▶ Aggregate predictions
- Works best with high-variance models (e.g., trees)

Bagging: Bootstrap Aggregation II

- Effect on variance:
 - ▶ Averaging reduces prediction variance
 - ▶ Assumes base learners are not perfectly correlated
- Visual: multiple noisy curves vs smooth averaged curve

Bagging

- Also called **Bootstrap aggregating**
- Can be performed in *parallel*
- Each iteration does not depend on the previous ones
- Generate N datasets using bootstrapping
- Use each dataset to train a model
- Combine the results of all the models to make a prediction

Bagging

- Suppose we are trying to perform a regression where the true outcome is $r(x)$
- Suppose each model gives us the true outcome plus an error:
 $m_i(x) = r(x) + \epsilon_i(x)$
- The error of each model is then $\mathbb{E}((m_i(x) - r(x))^2) = \mathbb{E}(\epsilon_i(x)^2)$

Bagging

Average error of the models acting individually

$$\mathbb{E}_{AV} = \frac{1}{M} \sum_{i=1}^M \mathbb{E}(\epsilon_i(x)^2)$$

Expected error from the ensemble

$$\begin{aligned} \mathbb{E}_{ENS} &= \mathbb{E}\left(\left(\frac{1}{M} \sum_{i=1}^M (m_i(x) - r(x))\right)^2\right) \\ &= \mathbb{E}\left(\left(\frac{1}{M} \sum_{i=1}^M \epsilon_i(x)\right)^2\right) \end{aligned}$$

Bagging

If we assume that errors are not correlated

$$\mathbb{E}(\epsilon_i(x) \cdot \epsilon_j(x)) = 0, \forall i \neq j$$

We can simplify

$$\begin{aligned}\mathbb{E}_{ENS} &= \frac{1}{M^2} \mathbb{E}\left(\left(\sum_{i=1}^M \epsilon_i(x)\right)^2\right) \\ &= \frac{1}{M^2} \sum_{i=1}^M \sum_{j=1}^M \mathbb{E}(\epsilon_i(x) \cdot \epsilon_j(x)) \\ &= \frac{1}{M^2} \sum_{i=1}^M \mathbb{E}(\epsilon_i(x)^2) \\ &= \frac{1}{M} \mathbb{E}_{AV}\end{aligned}$$

Bagging

- The average error of an ensemble can be reduced by a factor of M by averaging M versions of the model.
- But this is **only** true **if** all the individual errors are **uncorrelated**!
- In practice, the errors are usually highly correlated and thus the overall error reduction is small.

Section 5

Random Forest

Random Forests I

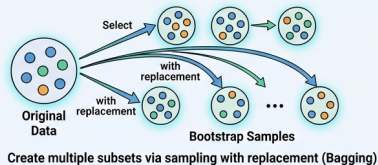
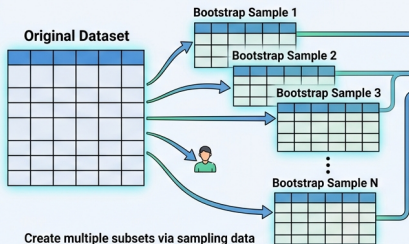
- Decision trees as base learners
- Additional randomness:
 - ▶ Random subset of features at each split
- Aim: decorrelate trees to improve ensemble performance

Random Forests II

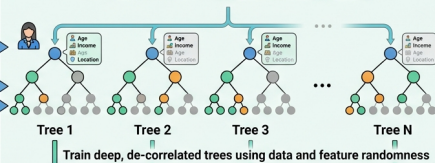
- Key hyperparameters:
 - ▶ Number of trees
 - ▶ Max features per split
 - ▶ Max depth / min samples per leaf
- Pros:
 - ▶ Strong baseline
 - ▶ Good default for tabular data

Random Forests III

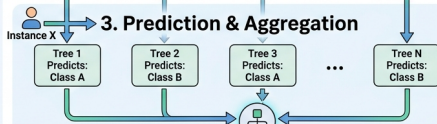
1. Training Data & Bootstrapping



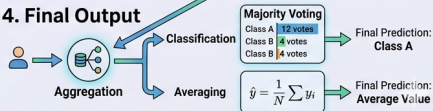
2. Random Forest Generation



3. Prediction & Aggregation



4. Final Output



What is a Random Forest?

At its core, a **Random Forest** is an ensemble of Decision Trees. Instead of relying on one “expert” tree, we consult an entire forest.

- **The Problem:** Single decision trees are prone to **overfitting** (they memorize the training data too well).
- **The Solution:** Build many diverse trees and average their results to reduce variance.
- **The Result:** A robust model that generalizes well to new, unseen data.

Step 1: Bagging (Bootstrap Aggregating)

The first layer of randomness is how we feed data to the trees. We use a technique called **Bagging**.

The Bootstrapping Process

- 1 Suppose we have a dataset of size N .
- 2 For each tree, we create a new training set by sampling N observations **with replacement**.
- 3 **Result:** Some rows appear multiple times in a tree's training set, while others (roughly 33%) are left out entirely (called **Out-of-Bag** data). *This ensures that no single outlier or data point can dominate the entire forest.*

Step 2: Building the Trees

When a standard Decision Tree splits a node, it looks at **every available feature** to find the best one. Random Forests change this.

Feature Randomness

- At every split in every tree, the algorithm chooses a **random subset** of features (columns).
- If you have p features, the forest might only look at $m = \sqrt{p}$ features for that specific split.
- **Why?** This prevents “strong” features from dominating every tree, forcing the forest to discover patterns in “weaker” features.

Step 3: Aggregation (The “Forest” Part)

Once all trees are grown, we have a collection of independent models. To get a final answer, we must **aggregate** their predictions.

For Classification

- Each tree “votes” for a category (e.g., “Email is Spam” vs “Not Spam”).
- The forest chooses the class with the **majority vote**.

For Regression

- Each tree predicts a numerical value (e.g., the price of a house).
- The final prediction is the **average** of all tree outputs:

$$\hat{y} = \frac{1}{T} \sum_{i=1}^T f_i(x)$$

Summary: Why it Works

The “magic” of Random Forests comes from **De-correlation**.

Technique	Purpose
Bootstrapping	Ensures trees see different rows of data.
Feature Randomness	Ensures trees look at different attributes.
Aggregation	Cancels out individual tree errors through averaging

Final Result: A highly accurate, stable, and “hard to break” machine learning model.

Section 6

Boosting

Boosting: Core Idea

- Sequential learning:
 - ▶ Each new model focuses on mistakes of previous ones
- Typically uses weak learners (shallow trees)
- Main effect: reduces bias, can increase variance

AdaBoost

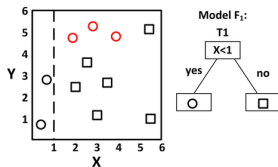
- Reweights training examples:
 - ▶ Higher weight for misclassified samples
- Combines weak learners with weighted voting
- Works best with relatively clean data, less noise

Gradient Boosting

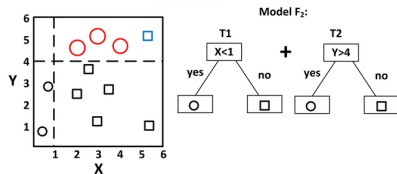
- In each iteration train model on the *residues* of the current model, i.e., it is trained to learn the errors
- Predictions are given by the sum of the predictions of all the models
- Can be applied to several models and is quite popular with decision trees
- Normally used with *shrinkage*, i.e., a multiplicative factor that reduces the contributions of each model in order to avoid overfitting
- Generalization of boosting:
 - ▶ Fits new learners to negative gradient of loss
- Supports arbitrary differentiable loss functions
- View as stage-wise additive model

Gradient Boosting

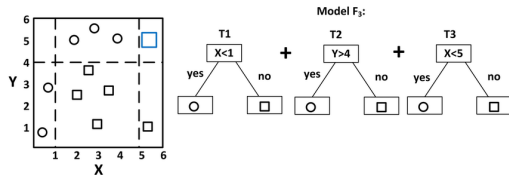
Iteration 1



Iteration 2



Iteration 3



Modern Boosting Methods

- XGBoost:
 - ▶ Regularization, shrinkage, advanced tree growth
- LightGBM, CatBoost:
 - ▶ Efficiency, handling categorical variables
- Widely used in ML competitions and industry

Bagging vs Boosting

Aspect	Bagging	Boosting
Learning	Parallel	Sequential
Main effect	Reduce variance	Reduce bias
Overfitting	Less prone	More prone if over-tuned
Parallelism	Easy to parallelize	Harder to parallelize

Boosting: Core Concepts

- **Philosophy:** Combine a sequence of “weak” learners into a “strong” ensemble.
- **Sequential Learning:** Each model $h_t(x)$ attempts to correct the errors of its predecessor $h_{t-1}(x)$.
- **General Framework:**

$$F_T(x) = \sum_{t=1}^T \alpha_t h_t(x)$$

- **Contrast:** Unlike Bagging (parallel/independent), Boosting is iterative and adaptive. Models are trained one after another, each correcting the errors of the previous ones.

AdaBoost (Adaptive Boosting)

- **Strategy:** Iteratively re-weight training samples based on classification difficulty.
- **Mechanism:**
 - 1 Initialize weights $w_i = 1/N$.
 - 2 Train weak learner $h_t(x)$.
 - 3 Calculate error $\epsilon_t = \sum w_i \cdot \mathbb{I}(y_i \neq h_t(x_i))$.
 - 4 Update weights: $w_{new} \propto w_{old} \cdot \exp(\alpha_t \cdot \mathbb{I}(y_i \neq h_t(x_i)))$.

AdaBoost (Adaptive Boosting)

- α_t gives greater weights to the more accurate classifiers.
- The first classifier uses the same weight for all examples; it is the same as the normal procedure.
- The weighting coefficients are increased for data points that are misclassified and decreased for data points that are correctly classified.
- Classifiers are forced to give greater emphasis to data points that were incorrectly classified

Gradient Boosting Machines (GBM)

- **Strategy:** Treat boosting as gradient descent in function space.
- **Optimization:** Minimize loss $L(y, F(x))$ by fitting models to negative gradients.
- **Update Step:**

$$F_t(x) = F_{t-1}(x) + \eta \cdot h_t(x)$$

where $h_t(x)$ is trained on pseudo-residuals:

$$r_{it} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{t-1}}$$

XGBoost (Extreme Gradient Boosting)

- **Improvements:** Incorporates regularization to prevent overfitting.
- **Regularized Objective:**

$$\mathcal{L}(\phi) = \sum_i L(y_i, \hat{y}_i) + \sum_k \Omega(f_k)$$

with complexity term $\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$.

- **Key Specs:** Second-order Taylor expansion (Hessians) for faster convergence.

LightGBM

- **Key Innovation:** Designed for high-dimensional, massive datasets.
- **Histogram-based:** Discretizes continuous features into bins.
- **GOSS (Gradient-based One-Side Sampling):** Retains high-gradient instances; subsamples low-gradient ones.
- **Leaf-wise Growth:** Prioritizes splitting the leaf that reduces loss the most, unlike the level-wise approach in XGBoost.
- **Efficiency:** Dramatically faster training and lower memory consumption on massive datasets.

AdaBoost in Python

SAMME is a multi-class implementation of AdaBoost

```
from sklearn.ensemble import AdaBoostClassifier
clf = AdaBoostClassifier(n_estimators=4,
                        random_state=0, algorithm='SAMME')
clf.fit(X, Y)
```

Gradient Boosting in Python

for regression

```
from sklearn.ensemble import GradientBoostingRegressor
model = GradientBoostingRegressor(n_estimators=3,
    learning_rate=1)
model.fit(X,Y)
```

for classification

```
from sklearn.ensemble import GradientBoostingClassifier
model = GradientBoostingClassifier()
model.fit(X,Y)
```

XGBoost in Python

```
from sklearn.model_selection import GridSearchCV
from sklearn.datasets import fetch_california_housing
import xgboost as xgb
import multiprocessing

X, y = fetch_california_housing(return_X_y=True)
xgb_model = xgb.XGBRegressor(
    n_jobs=multiprocessing.cpu_count() // 2)
clf = GridSearchCV(xgb_model, dict(
    max_depth = [2, 4, 6],
    n_estimators = [50, 100, 200]), n_jobs=2)
res = clf.fit(X, y)
res.best_score_
res.best_params_
```

Summary Comparison

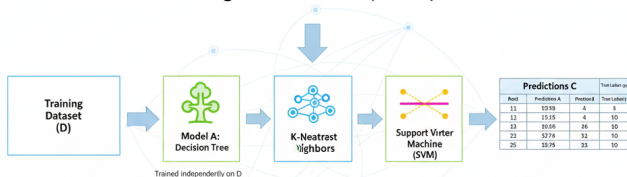
Feature	AdaBoost	GBM	XGBoost	LightGBM
Optimization	Sample Weights	Gradient	2nd Order	Histograms
Growth	Level-wise	Level-wise	Level-wise	Leaf-wise
Best For	Small/Med	General	Competitive	Very Large

Section 7

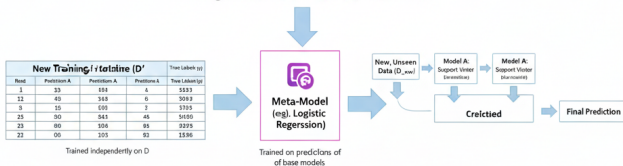
Stacking

ENSEMBLE LEARNING: STACKING (STACKED GENERALIZATION)

Stage 1: Base Models (Level 0)



Stage 2: Meta-Model (Level 1')



Ensemble Learning: Stacking

- **Concept:** Combine diverse base models (Level 0) by using their predictions as inputs to a meta-model (Level 1).
- **Workflow:**
 - 1 Train multiple diverse Level 0 models on the training set D_{train} .
 - 2 Generate predictions from these models.
 - 3 Create a new dataset D' where features are the predictions from Level 0 models and the target remains y .
 - 4 Train a Meta-Model on D' to make the final prediction.

Stacking: A Concrete Example

- **Level 0 (Base Models):**

- ▶ Train Decision Tree (h_1), KNN (h_2), and SVM (h_3) on D_{train} .

- **Feature Engineering:**

- ▶ Create new features $X'_i = [h_1(x_i), h_2(x_i), h_3(x_i)]$.

- **Meta-Model (Level 1):**

- ▶ Train a Logistic Regression model on X' to output the final classification.

- **Benefit:** The meta-model learns how to best weight the strengths/weaknesses of the base models.

Stacking vs. Blending

- **Stacking:** Uses Cross-Validation (K-fold) to generate “out-of-fold” predictions to train the meta-model, ensuring the meta-model sees predictions that aren’t overfitted to the training data.
- **Blending:** A simpler, faster alternative to stacking.
 - ▶ Holds out a small portion of the training set (a validation set).
 - ▶ Base models are trained on the remaining training data.
 - ▶ The meta-model is trained *only* on the predictions of the base models for the held-out validation set.

Understanding Blending

- **Process:**

- ① Split D_{train} into $D_{train_partial}$ and D_{val} .
- ② Train Base Models on $D_{train_partial}$.
- ③ Make predictions for D_{val} and D_{test} using the trained base models.
- ④ Train the Meta-Model using the predictions of D_{val} as the input features.
- ⑤ Final prediction is made by passing D_{test} predictions into the trained Meta-Model.

Pros and Cons

Feature	Stacking	Blending
Complexity	High (requires K-fold)	Low (simpler hold-out)
Data Usage	Efficient (uses all data)	Potential data waste
Overfitting	Robust (via CV)	Risk of overfitting to val set
Execution	Slower	Very Fast

Stacking I

- Idea: learn how to combine different models
- Level-0 models: base learners (e.g., RF, SVM, NN)
- Level-1 model: meta-learner trained on base outputs

Stacking II

- Implementation sketch:
 - ▶ Use cross-validation to generate out-of-fold predictions
 - ▶ Train meta-learner on these predictions
- Example:
 - ▶ Meta-learner: logistic regression
 - ▶ Base learners: random forest, SVM, gradient boosting

Blending

- Simpler alternative to stacking:
 - ▶ Use a hold-out validation set to train meta-learner
- Easier but less data-efficient
- Often used in quick competition setups

Practical Considerations I

- When ensembles shine:
 - ▶ Tabular data
 - ▶ Unstable base learners (trees)
- When they may not help:
 - ▶ Very small datasets
 - ▶ Highly correlated base models

Practical Considerations II

- Trade-offs:
 - ▶ Accuracy vs interpretability
 - ▶ Performance vs computational cost
- Model explainability:
 - ▶ Feature importance in Random Forests / boosting
 - ▶ Use of SHAP / permutation importance (mention briefly)

Section 8

Meta Model Selection

Choosing the Meta-Model (Level 1)

- **Goal:** The meta-model must learn how to combine (or “weight”) the predictions of base models without overfitting.
- **Core Rule:** Simplicity is a virtue. Complex meta-models often overfit the output noise of base models.
- **Default Choice:** Linear models (Logistic/Linear Regression).
 - ▶ Why? They are inherently regularized and handle highly correlated base-model predictions well.

When to Use Complex Meta-Models

- **Context:** Only use non-linear models (e.g., Shallow Decision Trees, Gradient Boosting) if there are complex interactions between base models.
- **Risk:** High risk of overfitting.
- **Mitigation:**
 - 1 Use strong regularization (e.g., shallow trees, Lasso/Ridge).
 - 2 Strictly rely on **Cross-Validation (Stacking)** to ensure the meta-model sees unseen “out-of-fold” predictions.

Meta-Model Selection Summary

Base Model Type	Objective	Recommended Meta-Model
Diverse (DT, SVM, KNN)	Robustness	Logistic/Linear Regression
Homogeneous (All Trees)	Accuracy	Ridge/Lasso Regression
High Variance	Stability	ElasticNet
Non-linear Errors	Prediction Power	Shallow Random Forest

Implementation Example (Python)

```
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression

# Define base models
level0 = [('dt', DecisionTreeClassifier()), ('svm', SVC())]

# Define meta-model
level1 = LogisticRegression()

# Combine into StackingClassifier
model = StackingClassifier(estimators=level0,
                           final_estimator=level1)
model.fit(X_train, y_train)
```

Section 9

Case Study

Case Study

- Example dataset:
 - ▶ Titanic (classification) or house prices (regression)
- Compare:
 - ▶ Single decision tree
 - ▶ Random Forest
 - ▶ Gradient boosting / XGBoost
- Show table or bar chart of metrics (accuracy/RMSE)

Section 10

Class Imbalance

The Challenge of Class Imbalance

- **Problem:** Standard algorithms assume balanced classes. When one class is rare ($\sim 10\%$), the model tends to ignore it (high bias).
- **Metrics:** Accuracy is misleading; focus on Precision, Recall, and the F1-Score instead.
- **Solution Strategy:** Manipulate the training data distribution before the model sees it.

Under-Sampling (The Majority Class)

- **Goal:** Reduce the number of majority class instances to match the minority class.
- **Techniques:**
 - ▶ **Random Under-Sampling:** Randomly remove instances from the majority class.
 - ▶ **Tomek Links:** Remove instances of the majority class that are very close to the decision boundary of the minority class.
- **Pro:** Reduces training time and memory usage.
- **Con:** Significant information loss (data is discarded).

Over-Sampling (The Minority Class)

- **Goal:** Increase the number of minority class instances.
- **Techniques:**
 - ▶ **Random Over-Sampling:** Simply duplicate existing minority samples (leads to overfitting).
 - ▶ **SMOTE (Synthetic Minority Over-sampling Technique):** Generate new, synthetic examples by interpolating between existing minority samples.
 - ▶ **ADASYN:** Similar to SMOTE, but focuses on generating synthetic data in areas where the model struggles (high-density/border regions).

Comparison: Under vs. Over-Sampling

Technique	Best Used For	Primary Risk
Under-sampling	Very large datasets	Loss of valuable information
Random Over-sampling	Small datasets	Severe Overfitting
SMOTE	Moderate imbalance	Over-generalization

Best Practices

- **Never Resample Test Data:** Always perform resampling *only* on the training split. Resampling the test set leads to biased evaluation metrics.
- **Validation Strategy:** Use Stratified K-Fold Cross-Validation to ensure each fold maintains the original class distribution.
- **Combine with Ensembles:** Resampling often works best when combined with boosting algorithms (e.g., EasyEnsemble or Balanced Random Forest).

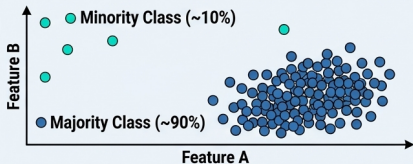
Section 11

SMOTE

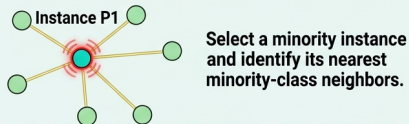
SMOTE

EXPLANATION OF SMOTE: SYNTHETIC MINORITY OVER-SAMPLING TECHNIQUE

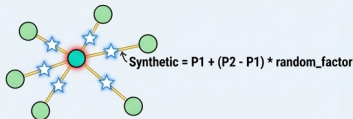
1. Imbalanced Dataset (Initial State)



2. Find Minority Class Neighbors

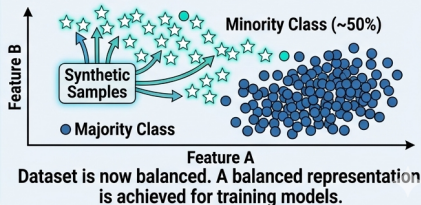


3. Generate Synthetic Data Points



New synthetic instances are created on the lines between the instance and its neighbors.

4. Balanced Dataset (Resulting State)



SMOTE: Synthetic Minority Over-sampling

- **Objective:** Instead of simple duplication, SMOTE generates new, plausible samples by interpolating between existing minority class data points.
- **The Process:**
 - 1 Select a minority class instance x_i .
 - 2 Identify its k -nearest neighbors of the same class.
 - 3 Select one neighbor x_{zi} at random.
 - 4 Generate a synthetic point x_{new} along the line connecting x_i and x_{zi} .

The Interpolation Formula

- **Mathematical Representation:**

$$x_{new} = x_i + \lambda \cdot (x_{zi} - x_i)$$

where $\lambda \in [0, 1]$ is a random number generated for each new synthetic sample.

Why This Works

- **Avoids Overfitting:** Unlike random over-sampling, it creates *unique* points rather than duplicates.
- **Softens Decision Boundaries:** By populating the space between minority samples, it prevents the model from forming highly localized, overfitted decision boundaries around original points.
- **Limitations:**
 - ▶ It does not consider the majority class distribution.
 - ▶ It may lead to “overlap” if minority points are too close to majority clusters, increasing the risk of misclassification.

Implementation Tip: The Pipeline

- **Crucial:** Apply SMOTE **only** within your training cross-validation folds.
- Never apply it to the test set, as this would cause “data leakage” (the model would be tested on data generated based on information present in the training set).

Section 12

Practical Ensemble Learning

When to Choose Ensembles?

- **Base Learner Stability:** Ensembles excel when base learners are “unstable”—meaning small changes in the training data lead to significantly different models (e.g., Deep Decision Trees).
- **Performance Ceiling:** Use ensembles when a single model (e.g., Logistic Regression) has hit a performance plateau and you need marginal gains in accuracy.
- **The “No-Free-Lunch” Rule:** If a simple linear model performs well enough, the added complexity of an ensemble is often not justified.

Computational Cost vs. Performance

- **The Trade-off:** Increasing accuracy via ensembles often incurs a linear or exponential cost in training/inference time.
- **Real-World Pipeline Considerations:**
 - ▶ **Training Latency:** Sequential models (Boosting) are inherently harder to parallelize than Bagging (Random Forest).
 - ▶ **Inference Latency:** A stack of 50 models requires 50 forward passes. Use model distillation if real-time inference is required.

The Overfitting vs. Interpretability Trade-off

- **Interpretability:** Ensembles are notoriously “Black Box.”
 - ▶ A single Decision Tree is transparent.
 - ▶ An XGBoost model with 1,000 trees is not.
- **Overfitting:** Stacking meta-models are prone to overfitting if they are too complex.
- **Regulation:** Always use shrinkage (learning rate) and tree depth constraints to curb overfitting in boosting.

Feature Importance & Explainability

- **Inherent Metrics:** Boosting models provide “Gain” or “Split” importance, but these can be biased toward high-cardinality features.
- **Model-Agnostic Tools:**
 - ▶ **SHAP (SHapley Additive exPlanations):** Standard for deep explainability in ensembles.
 - ▶ **Permutation Importance:** Measures the drop in model score when a feature is shuffled.
- **Consistency:** Ensure feature importance aligns with domain expertise.

Summary: The Pipeline Checklist

Factor	Recommendation
Data Size	Large → LightGBM/XGBoost
Real-time Req	Low latency → Linear / Single Tree
Accuracy Req	High → Stacking / Deep Boosting
Explainability	High → Linear Model / SHAP Analysis